# Sistema di Gestione per un Negozio di Alimentari

Autore: Gino Pandozzi-Trani

Luglio 2025

# Contents

**1 Introduzione** **3**

**2 Analisi dei Requisiti** **4**

**3 Progettazione Concettuale** **5**
3.1 Diagramma ER . . . . . 5
3.2 Descrizione delle entità . . . . . 5

**4 Progettazione Logica** **6**
4.1 Schema relazionale . . . . . 6
4.2 Dizionario delle Classi, delle Associazioni e dei Vincoli . . . . . 6
4.3 Schema Logico: Trigger e Procedure . . . . . 6

**5 Progettazione Fisica** **7**
5.1 Script SQL . . . . . 7
5.2 Trigger, Funzioni e View . . . . . 7

**6 Popolamento e Dati di Esempio** **8**

**7 Query e Funzionalità Implementate** **9**

# 1 Introduzione

SISTEMA DI GESTIONE PER UN NEGOZIO DI ALIMENTARI Si sviluppi un sistema informativo, composto da una base di dati relazionale e un applicativo Java dotato di GUI (Swing o JavaFX), per la gestione di un sistema di un negozio di ortofrutta. Il sistema deve permettere di gestire le vendite e la disponibilità dei prodotti disponibili, identificando le tipologie (frutta, verdura, farinacei, latticini, uova e confezionati). Ogni tipologia di prodotto ha delle caratteristiche diverse (ad esempio, la frutta e la verdura hanno una data di raccolta, i latticini hanno sia una data di mungitura del latte che una data di produzione, ecc..). Ogni cliente è registrato e ha una tessera punti associata che lo identifica al fruttivendolo. Per ogni acquisto il cliente riceve il 10% del valore della spesa in punti fedeltà. Il sistema deve premettere la ricerca dei clienti, differenziandoli sulla base delle categorie di prodotti acquistati e sulla quantità di punti che hanno ottenuto per ogni categoria. Per il gruppo da 3: il fruttivendolo ha un certo numero di dipendenti, e ad ogni vendita è associato il dipendente che l'ha portata a termine. Deve pertanto essere possibile ricercare quale dipendente ha effettuato il maggior numero di vendite in un determinato periodo differenziandolo dal dipendente che ha portato al fruttivendolo il maggior introito. Breve presentazione del progetto, obiettivi e contesto della traccia accademica.

# 2 Analisi dei Requisiti

- Gestione prodotti per tipologia (frutta, verdura, farinacei, latticini, uova, confezionati)
- Gestione clienti e tessere punti
- Gestione dipendenti e vendite
- Statistiche e ricerche

# 3 Progettazione Concettuale

## 3.1 Diagramma ER

## 3.2 Descrizione delle entità

# 4 Progettazione Logica

## 4.1 Schema relazionale

Lo schema relazionale è stato progettato per rappresentare tutte le entità e le relazioni individuate nell'analisi. Comprende tabelle per prodotti, clienti, dipendenti, vendite, movimenti punti e categorie di prodotto. Sono stati definiti tipi ENUM per le tipologie di prodotto e vincoli di integrità referenziale per garantire la coerenza dei dati.

## 4.2 Dizionario delle Classi, delle Associazioni e dei Vincoli

| Classe (Tabella) | Descrizione |
|---|---|
| cliente | Dati anagrafici dei clienti. |
| tessera | Tessera punti associata a ciascun cliente, saldo punti e stato. |
| dipendente | Dati anagrafici dei dipendenti. |
| prodotto | Prodotti in vendita, tipologia, caratteristiche e scorta. |
| ordine | Vendita, associata a cliente e dipendente. |
| articoliordine | Dettaglio dei prodotti acquistati in ogni ordine. |

Table 1: Classi principali del database

| Associazione | Descrizione |
|---|---|
| Cliente-Tessera | Ogni cliente ha una tessera (1:1). |
| Ordine-Cliente | Ogni ordine è associato a un cliente (N:1). |
| Ordine-Dipendente | Ogni ordine è associato a un dipendente (N:1). |
| Ordine-Prodotto | Ogni ordine contiene più prodotti tramite articoliordine (N:M). |

Table 2: Associazioni tra le entità

| Vincolo | Descrizione |
|---|---|
| Chiavi primarie | Ogni tabella ha una chiave primaria. |
| Chiavi esterne | Vincoli tra tessera-cliente, ordine-cliente, ordine-dipendente, articoliordine-prodotto, articoliordine-ordine. |
| Vincoli CHECK | Su formati (email, codice fiscale), valori (scorta, prezzo), e coerenza campi prodotto per categoria. |

Table 3: Vincoli principali

## 4.3 Schema Logico: Trigger e Procedure

| Trigger/Procedura | Descrizione |
|---|---|
| VerificaScortaProdotto | Blocca l'inserimento di articoli in ordine se la scorta è insufficiente. |
| AggiornaScortaEPunti | Aggiorna la scorta del prodotto e accredita i punti fedeltà al cliente dopo l'inserimento di un articolo. |
| AggiornaPrezzoOrdine | Aggiorna il prezzo totale dell'ordine dopo l'inserimento di un articolo. |
| AggiornaStatoTessera | Aggiorna automaticamente lo stato della tessera in base alla data di scadenza. |
| RipristinaScortaProdotto | Ripristina la scorta e ricalcola i punti dopo la cancellazione di un articolo da un ordine. |
| AggiornaPrezzoOrdineDelete | Aggiorna il prezzo totale dell'ordine dopo la cancellazione di un articolo. |

Table 4: Trigger e procedure principali implementate

# 5 Progettazione Fisica

## 5.1 Script SQL

Gli script SQL sono suddivisi per facilitare la gestione e la manutenzione del database. In particolare:

- `Tabelle.sql`: definizione delle tabelle e dei vincoli.
- `Funzioni.sql`: implementazione di funzioni e procedure per operazioni ricorrenti.
- `Trigger.sql`: automazione di controlli e aggiornamenti tramite trigger.
- `Ricerche.sql`: query avanzate e viste per le principali funzionalità applicative.

Le scelte implementative privilegiano la chiarezza e la modularità.

## 5.2 Trigger, Funzioni e View

Sono stati implementati diversi trigger per automatizzare aggiornamenti (ad esempio, l'assegnazione automatica dei punti fedeltà dopo una vendita) e garantire la coerenza dei dati. Le funzioni SQL supportano operazioni come il calcolo dei punti, la ricerca dei migliori dipendenti e la gestione delle statistiche. Le view facilitano l'accesso ai dati aggregati e alle informazioni più richieste dall'applicativo.

# 6 Popolamento e Dati di Esempio

Il popolamento del database è stato effettuato tramite lo script `Popolazione.sql`, che inserisce dati di esempio per tutte le principali entità del sistema:

- Prodotti di tutte le tipologie (frutta, verdura, farinacei, latticini, uova, confezionati), con relative caratteristiche specifiche.
- Clienti con tessere punti associate e saldo iniziale.
- Dipendenti e dati anagrafici.
- Vendite, ciascuna associata a cliente, dipendente e prodotti acquistati.
- Movimenti punti fedeltà generati dalle vendite.

I dati inseriti permettono di testare tutte le funzionalità implementate, comprese le query di ricerca, le statistiche e la gestione dei punti fedeltà. Lo script può essere eseguito dopo la creazione delle tabelle per ottenere un database pronto all'uso con dati realistici.

# 7 Query e Funzionalità Implementate

Le query implementate permettono di soddisfare tutte le richieste funzionali del sistema, tra cui:

- Ricerca clienti per categoria di prodotti acquistati e punti accumulati.
- Statistiche sulle vendite e sui dipendenti (numero vendite, introiti, ecc.).
- Gestione e consultazione dei punti fedeltà.
- Analisi delle disponibilità dei prodotti per tipologia.

Ogni query è stata progettata per essere efficiente e facilmente integrabile nell'applicativo Java.